応用情報技術者試験　テクノロジ系　#5

ソフトウェア

用語集まとめ ― 過去問10回分（令和3〜7年度）で出たところを中心に



全体像 シラバスの5つの小分類を、5章に分けて読む

1 OSの種類と構成
基本ソフトウェア・カーネル・動作モード・ジョブ管理・スプーリング・起動・アクセス制御

2 タスク管理
3つの状態・プリエンプション・スケジューリング・多重度・排他制御・デッドロック

3 記憶管理
区画方式・断片化・仮想記憶・ページフォールト・ページ置換え（FIFO・LRU）

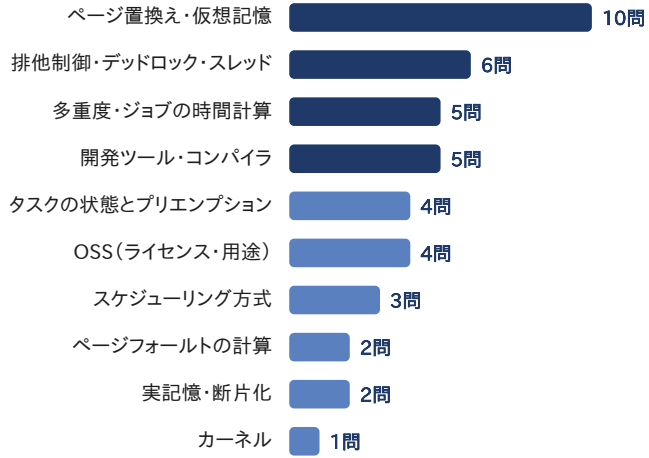
4 ミドルウェアとファイルシステム
TPモニター・API・ライブラリ・シェル・パス・リンク・バックアップ

5 開発ツールとOSS
コンパイラの処理・リンカとローダ・デバッグの道具・Git・ライセンス・用途別のOSS

午前での出方	
問題数	毎回 **問16〜問20 あたりで4問前後**（R7春は問14〜17）。10回で42問
同じ問題の再出題	5組。プリエンプションの遷移先・多重度の計算・デッドロックの防止・スラッシング・Git。I2Cのセマフォは選択肢を1つ変えて再出題
問題の型	**時間の線や表を書いて数える計算が約3分の1**（ターンアラウンド・多重度・ページ置換え）。残りは用語の入れ替えを見抜く問題

この資料の読み方：1枚に1つのメッセージ。計算の問題は、図と同じ線や表を自分で書けば解けるようにしてある。

4分の1は仮想記憶。ページの出し入れを表に書ければ取れる



令和3年度春期〜令和7年度秋期の42問を、論点ごとに数えた（同じ問題も1問ずつ数えた）

論点	押さえること
ページ置換え	FIFO＝読み込んだ時刻、LRU＝最後に使った時刻、LFU＝回数。**見る列が違う**
仮想記憶の事象	ページフォールト＝ページイン ≧ ページアウト。急に遅くなるのはスラッシング
排他制御	共有資源はセマフォ。デッドロックは**資源を取る順番をそろえて**防ぐ
時間の計算	横軸に時間を取った線（ガントチャート）を書き、終了時刻−到着時刻
状態遷移	横取りされたら**実行可能状態**。待ち状態ではない
開発ツール・OSS	道具の名前と役目の組合せ。 GPL→LGPL→MPL→BSDの順に義務が軽い

第1章

OSの種類と構成

ソフトウェアの体系 ／ カーネル ／ CPUの動作モード ／ ジョブ管理 ／ スプーリング ／ 起動 ／ 利用者とアクセス制御



OSはハードウェアとアプリの間で、資源を配る係。中核がカーネル

応用ソフトウェア
販売管理・表計算・Webブラウザ

ミドルウェア
DBMS・TPモニター・Webサーバ・運用管理ツール（第4章）

OS（基本ソフトウェア）
制御プログラム（中心がカーネル）＋言語プロセッサ＋サービスプログラム

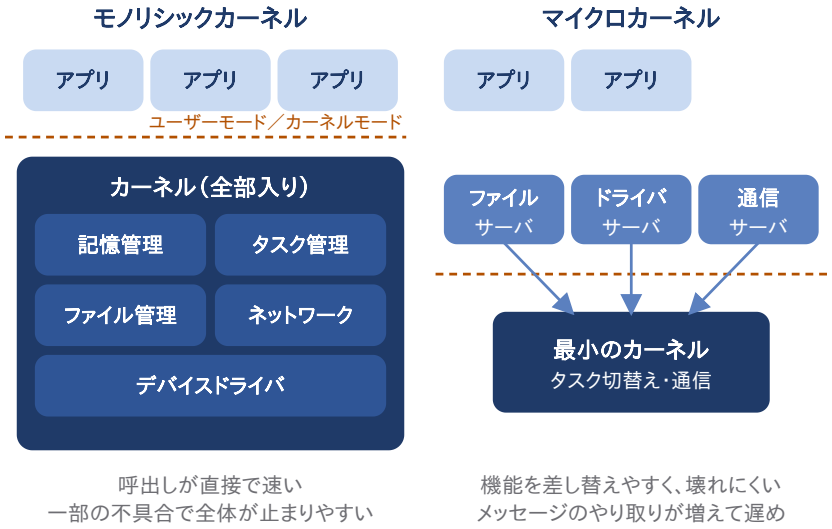
ハードウェア
CPU・主記憶・ディスク・入出力装置

OSとミドルウェアをまとめて**システムソフトウェア**と呼ぶ。サービスプログラム（整列・併合・アーカイバなど）は**ユーティリティ**とも呼ぶ。

OSの管理機能	受け持つこと
ジョブ管理	利用者の仕事（ジョブ）を受け付け、順番を決めて実行させる
タスク管理	実行の単位（タスク）にCPUを割り当てる（第2章）
記憶管理	主記憶を割り当て、仮想記憶を管理する（第3章）
データ管理	ディスク上のデータをファイルとして読み書きさせる
入出力管理	装置の割当てとスプーリング・バッファリング
運用管理・障害管理	起動と終了・利用者の管理・監視・障害の記録と回復

OSの種類：**リアルタイムOS**は締切りまでに応答することを最重視。**組込みOS**は機器に入る小さなOS。**セキュアOS**はrootの権限も細かく制限する。

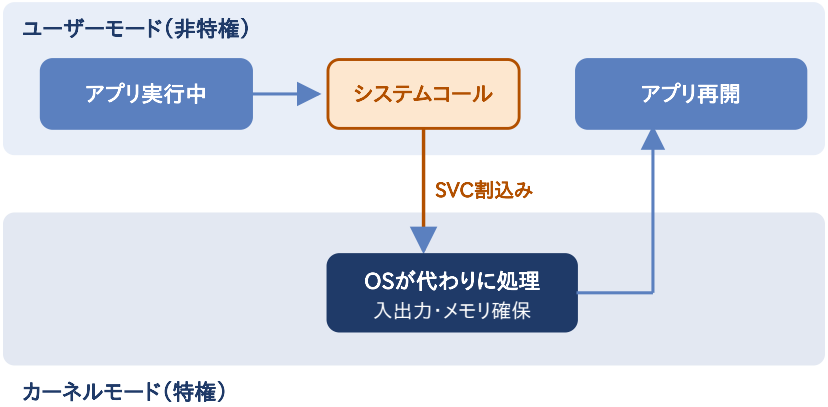
カーネルに何を入れるかで2通り。シェルもGUIもカーネルの外



ソフトウェア	カーネルの中か外か
プロセス管理・メモリ管理	**中**。アプリが動くための基本機能
シェル（bashなど）	外。コマンドを解釈してカーネルを呼ぶ
デスクトップ環境（GUI）	外。画面の操作を受け持つ別のソフト
ブラウザ・ワープロ	外。応用ソフトウェア

R5秋 問19「Linuxカーネルの説明」：正解は「プロセス管理やメモリ管理などの基本機能を提供する」。シェルやGUIが**組み込まれている**とする選択肢は誤り。Linuxはモノリシックカーネル。

🔑 アプリは特権命令を使えない。OSに頼むとカーネルモードに切り替わる



モード	動くもの	使える命令
カーネルモード（スーパーバイザー・特権）	OSのカーネル	全部。入出力・割込みの設定などの**特権命令**も
ユーザーモード（非特権）	アプリケーション	特権命令以外。主記憶も自分の範囲だけ

> アプリが自分でカーネルモードに上がることはできない。ユーザーモードで特権命令を実行すると**例外（プログラム割込み）**になり、実行されない。

> 目的は**保護**。アプリの誤りや悪意が、OSや他のプログラムを壊さないようにする。割込みの種類（SVC・入出力・タイマなど）は #3 にまとめた。

ジョブ→ジョブステップ→タスクの順に細かくなる。一時ファイルは表で数える

ジョブ
利用者の仕事
▶
ジョブステップ
プログラム1本の実行
▶
タスク
CPUを割り当てる単位

ジョブ管理の部品	役目
ジョブスケジューラー	リーダ（読込み）→イニシエータ（開始）→ターミネータ（後始末）→ライタ（結果の出力）
マスタスケジューラー	オペレーターとのやり取りと、システム全体の監視
JCL	ジョブの中身をOSに指示する言語

夜間にまとめて流す**バッチ処理**、定期実行の cron、裏で動かす**バックグラウンドジョブ**も、ジョブの動かし方の言葉。

R4春 問16：多重度2、各ジョブ50Mバイトの一時ファイル

時間の枠	動くジョブ	残っているファイル	合計
①	A	a	50
②	B・C	a（Bが参照）・b・c	150
③	D・E	b（Dが参照）・c（Eが参照）・d・e	**200**
④	F	d・e（Fが参照）・f	150

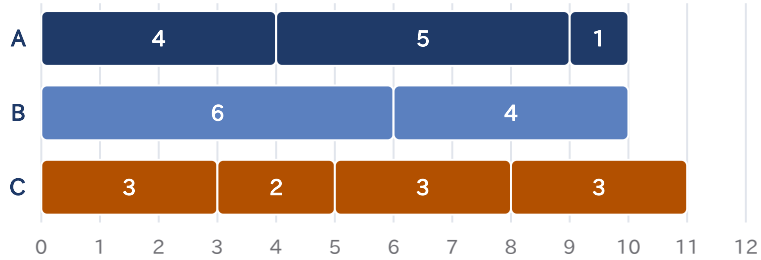
ファイルは「作ったジョブの**直後のジョブ**が終わるまで」残る。Bが終わるとaが消え、DとEが終わるとbとcが消える。最大の③が答え（**200Mバイト**）。

遅い装置への出力はディスクにためる。割当ては空いた時刻の順に追う

プログラム
出力して次へ進む
▶
ディスク
スプール（待ち行列）
▶
プリンタ
空いたら少しずつ印刷

仕組み	ためる場所と単位
スプーリング	ディスクに、ジョブ（印刷データ）の単位でためる。CPUが装置を待たない
バッファリング	主記憶のバッファに、入出力の単位でためて速度差を吸収
DMA（#3）	CPUを通さずに装置と主記憶の間で転送する

R4秋 問19：4,6,3,2,5,3,4,3,1 分を A→B→C の順に割当て（目盛りは分）



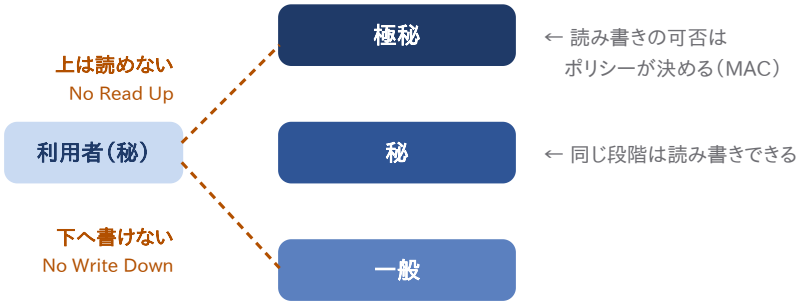
時刻0にA・B・Cへ。あとは**空いた時刻の早い順**に次のデータを渡す（3分にC、4分にA、5分にC、6分にB、8分にC、9分にA）。Cの合計は 3+2+3+3=**11分**。

起動は小さなプログラムからの順送り。アクセスは「誰が決めるか」で2種類



この順送りを**ブートストラップ（IPL）**と呼ぶ。サーバからOSを受け取るのが**ネットワークブート**、起動時にOSを選ぶのが**マルチブート**（同時に動くOSは1つ）。

アクセス制御	許可を決める人
任意（DAC）	ファイルの**所有者**が自分で決める。ふつうのOSのアクセス権
強制（MAC）	システムの**ポリシー**が決め、所有者でも変えられない。セキュアOS



BLPモデルは機密性を守る規則（上を読まない・下へ書かない）。利用者は**root（Administrator）**・一般ユーザー・ゲストに分け、ふだんは最小の権限で使う（WindowsのUAC）。アカウントの一元管理は**LDAP・Active Directory**。

第2章

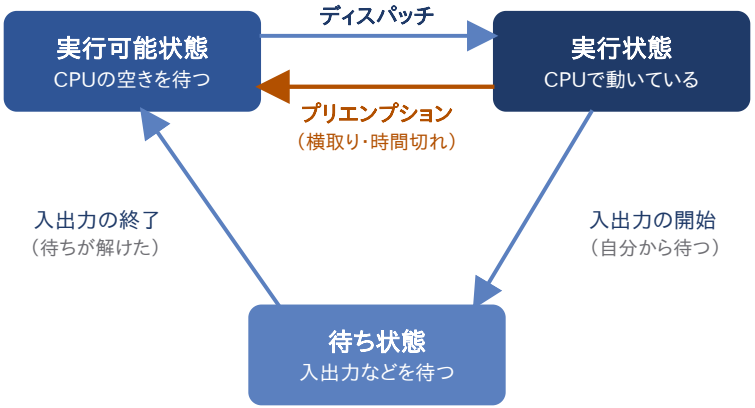
タスク管理

3つの状態 ／ プリエンプション ／ スケジューリング ／ ターンアラウンドタイム ／ 多重度 ／ 排他制御 ／ デッドロック






横取りされたタスクは「実行可能」に戻る。待ち状態ではない



待ち → 実行 には直接移らない（いったん実行可能へ）

状態	いつその状態か
実行状態	CPUを使って命令を実行している（CPU1個なら1つだけ）
実行可能状態	CPUさえあればすぐ動ける。待ち行列に並んでいる
待ち状態	入出力の終了・資源・イベントを待っていて、CPUが空いても動けない
休止状態（RTOS）	登録されているが起動前、または実行を終えた

R3春 問17・R7春 問14（同じ問題）：リアルタイムOSで、プリエンプションによって移る状態は**実行可能状態**。「待ち状態」「休止状態」「終了状態」は誤り。

🔔 OSが横取りできるのがプリエンプティブ。割込み処理はどのタスクよりも先

	プリエンプティブ	ノンプリエンプティブ
切り替わるとき	OSの判断で、いつでも（優先度の高いタスクの到着・時間切れ）	タスクが**自分から**手放したときだけ（終了・待ち・中断の要求）
無限ループのタスク	他のタスクは動ける	他のタスクが**動けなくなる**
使われる所	リアルタイムOS・今のPC用OS	昔のOS・単純な組込み機器

R6春 問16：ノンプリエンプティブは「自らの中断をOSに要求したときだけ切り替わる」。「優先度の高いタスクが来たら中断」はプリエンプティブの動き。

割込み処理ルーチン（割込みハンドラ）
どのタスクよりも先に実行（割込み禁止中を除く）

▼

優先度の高いタスク

▼

優先度の低いタスク

R4秋 問18：割込み処理ルーチンは**最も優先度の高いタスクよりも優先**して実行される。処理が長いと応答性が落ちるので短く作る。割り込まれたタスクとは**別のコンテキスト**で動き、タスク切替えを禁止しても割込みは止まらない。

⏳ 短い処理を優先すると、長い処理がいつまでも待たされることがある

方式	次に動かすタスク	横取り	押さえどころ
到着順（FCFS）	先に来たもの	しない	公平で簡単。長い処理の後ろに並んだ短い処理が待たされる
処理時間順（SJF）	処理時間が最も短いもの	しない	平均の待ち時間は最短。短いものが次々来ると、長いものが**待ち続ける（スタベーション）**
優先度順	優先度が最も高いもの	することが多い	低い優先度が待たされる。待つほど優先度を上げる**エージング**で防ぐ
ラウンドロビン	待ち行列の先頭	する（時間切れで最後尾へ）	全員に順番が回る。タイムクウォンタムが短すぎると切替えの手間が増える
多段フィードバック待ち行列	高い段の待ち行列の先頭	する（時間切れで一段下へ）	短い処理は早く終わる。処理時間を知らなくてもSJFに近い効果

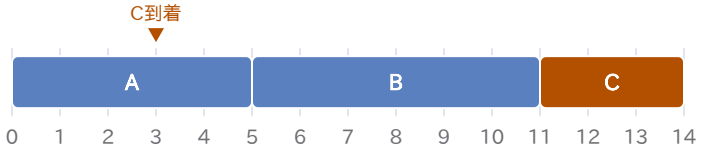
R6秋 問16：「特定のタスクがCPUの割当てを待ち続ける可能性が最も高い」のは**処理時間順（SJF）**。エージング付きの優先度順・ラウンドロビン・到着順は、待てばいつか順番が来る。



⏱️ ターンアラウンドタイムは「終了時刻ー到着時刻」。時間の線を書けば数えられる

R3春 問16:到着順・多重度1

ジョブ	到着	処理
A	0	5
B	2	6
C	3	3



Cは11秒に始まり14秒に終わる。**14−3=11秒**(待った8秒+実行3秒)。

R7秋 問15:処理時間順・多重度1

ジョブ	A	B	C	D	E
到着	0	1	2	3	4
処理	2	4	3	2	1



1つ終わるたびに、**その時点で到着済み**のものから最短を選ぶ。2秒:B・CからC。5秒:B・D・EからE→D→B。**12−1=11秒**。

多重度は「同時に動ける数」。要求の到着時刻をずらして線を引く

R4春 問19・R6春 問18（同じ問題）

多重度2以下なら1件4秒。1秒おきに4件届く。多重度1と2で、全部終わるまでの時間の差は？



多重度	考え方	終わる時刻
1	1件ずつ順に4秒	16秒
2	2本の列で並行。2件目は1秒に到着して1〜5秒、4件目は5秒に始まる	9秒

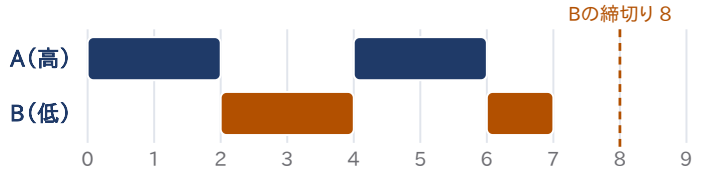
差は **16−9=7秒**。「4件÷2×4秒＝8秒」と割り算すると、到着が1秒ずつずれることを見落とす。

多重度を上げすぎると、主記憶が足りなくなりページの出し入ればかりになる（第3章のスラッシング）。

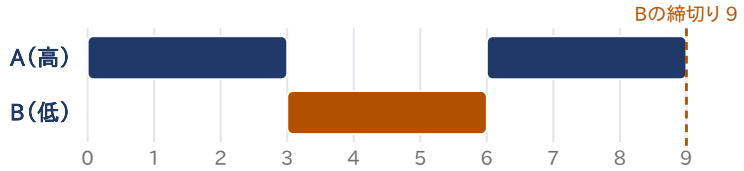
周期タスクは、高い方が使った残りの時間で低い方が終わるかを数える

R5秋 問17：Aの優先度が高い。Bは周期内に終わるか

ア A：2ミリ秒／周期4　B：3ミリ秒／周期8



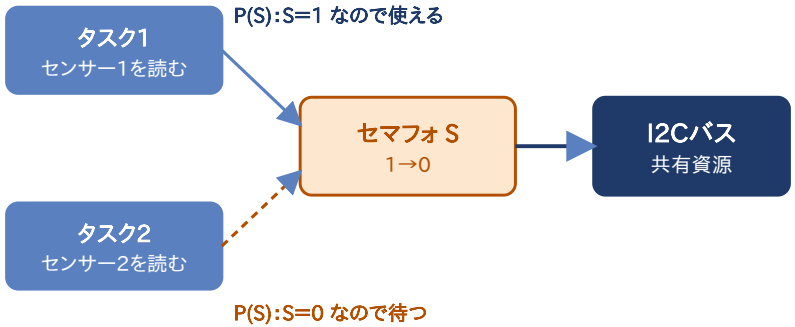
イ A：3ミリ秒／周期6　B：4ミリ秒／周期9



選択肢	Bの周期内でAが使う時間	残り	Bの必要
ア	2+2=4	8−4=4	3 ○
イ	3+3=6	9−6=3	4 ×
ウ	3+3+3=9	13−9=4	5 ×
エ	4+4+3=11	15−11=4	5 ×

Aは周期が来るたびに**Bを横取りして**先に動く（プリエンプティブ）。エは3回目のAが12〜16ミリ秒にかかるが、Bの締切り15までの3ミリ秒を数える。

共有資源を1つずつ使わせるのがセマフォ。キューやラウンドロビンは別の役

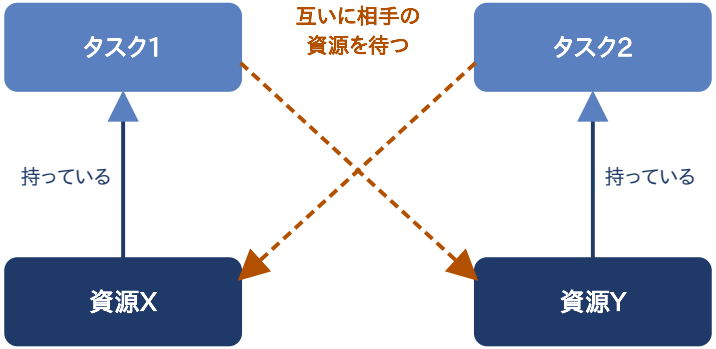


P操作：Sが1以上なら1減らして進む。0なら待つ
V操作：Sを1増やし、待っているタスクを起こす

R4春 問17・R7秋 問16 の選択肢	
セマフォ	**正解**。資源の空きを数で表し、P操作・V操作で確保と解放
キュー	タスク間でデータを順に受け渡す（メッセージキュー）
マルチスレッド	処理を並行して動かすこと。同時に**使わせない**仕組みではない
ラウンドロビン	CPUを順番に回すスケジューリングの方式
メモリプール	決まった大きさのメモリを貸し出す記憶管理（R7秋で入れ替わった選択肢）

ロックしたタスクだけが解放できる**ミューテックス**、ループで待つ**スピンロック**も排他制御の道具。守る部分を**クリティカルセクション**と呼ぶ。

デッドロックは待ちの輪。全員が同じ順番で資源を取れば輪ができない



取る順番が「X→Y」と「Y→X」で逆だと、待ちの輪ができる

R4秋 問16・R7秋 問18（同じ問題）：防ぐ方法は「資源獲得の順序を両方のタスクで**同じ**にする」。優先度を変えても防げない。

R6春 問17：Aと同時に動かしてデッドロックが起きうるのは？

プロセス	取る順（表の数字を並べ直す）	Aと
A	X → Y → Z	—
B	X → Y → Z	同じ順 ○
C	Z → X → Y	Xを持つAとZを持つCが**待ち合う**
D	Z → Y → X	XとYを逆に取り合う

答えは**C, D**。表の数字は「何番目に取るか」なので、まず資源を**取る順に並べ直す**のがこつ。



並行処理の困りごとは5つ。「止まる」「進まない」「結果が狂う」で分ける

現象	何が起きるか
デッドロック	互いに待って**全員が止まる**
ライブロック	譲り合いを繰り返し、動いているのに**進まない**
スタベーション	他は動くのに、**特定のタスクだけ**待たされ続ける
優先度逆転	低い優先度のタスクがロックを持つ間に、中くらいが動き続け、高いタスクが待たされる。**優先度継承**で防ぐ
競合状態	同時に読み書きして、タイミングで**結果が狂う**

競合状態の例：残高100に、2つのスレッドが10ずつ足す



正しくは120。片方の更新が消える

読む→足す→書く をクリティカルセクションとして排他制御する

R7秋 問17：スレッドセーフは「複数のスレッドで並列に実行しても問題が生じない」性質。「複数のスレッドから呼ばれないようにする」「単一スレッドで動かす」は誤り。

第3章

記憶管理

区画方式 ／ 断片化とGC ／ 仮想記憶 ／ ページングとセグメンテーション ／ ページフォールト ／ ページ置換え（FIFO・LRU）／ スラッシング



空きの合計は足りても、続いた空きがないと入らない。それが断片化



可変区画：ジョブごとに必要な大きさだけ

新しいジョブ 30

空きは 25・15・40
合計80あっても
入るのは 40 の所だけ

どの空きに入れるか

ファーストフィット：最初に見つかった入る空き

ベストフィット：入る中で一番小さい空き

ワーストフィット：一番大きい空き

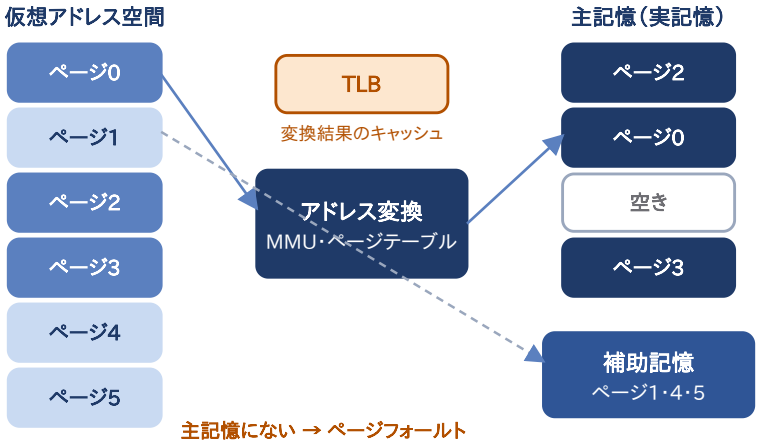
空きが細切れになるのが断片化

→ 詰め直す（コンパクション）
連続した大きな空きを作る

方式	しくみ	弱み
固定区画	主記憶を決まった大きさに区切る	区画の中に余りが出る（**内部断片化**）
可変区画	必要な大きさだけ切り出す	区画の間に細かい空きが出る（**外部断片化**）
オーバーレイ	プログラムを分け、必要な部分だけ順に読み込む	分け方を作る人が決める
スワッピング	実行中でないプログラムを丸ごとディスクへ出す	出し入れの時間がかかる

R4春 問18：断片化は「空き領域の合計は十分なのに、必要な大きさの**連続した領域**を獲得できない」状態。**R3春 問18**：使われなくなったヒープ領域を回収するのは**ガーベジコレクション**（コンパクションや動的再配置と入れ替えた選択肢に注意）。

仮想記憶は、主記憶より大きなプログラムを、必要なページだけ載せて動かす



プログラムは連続した広い空間に見える。実際は飛び飛びでも構わない

部品	役目
ページテーブル	仮想ページ番号→主記憶の枠番号の対応表。主記憶にあるかどうかの印も持つ
MMU	アドレス変換を行うハードウェア
TLB	最近の変換結果を覚えておく小さな高速メモリ。ページテーブルを毎回読まずに済む
補助記憶（ページング領域）	ページアウトされたページの置き場所

R3秋 問19：仮想記憶での補助記憶の役目は「**ページアウトされたページを格納する**」こと。更新前の内容の保存やバックアップではない。

ページは**必要になったときに読み込む**（デマンドページング）のがふつう。先に読んでおくのはプリページング。

📜 ページは固定長、セグメントは意味の単位。ページングは外部断片化なし

	ページング	セグメンテーション
分け方	**固定長**のページ	プログラム・データなど**論理的なまとまり**（可変長）
断片化	外部断片化は**起きない**（最後のページに内部断片化）	外部断片化が起きる
保護・共有	ページ単位なので意味と合わない	まとまりごとに保護や共用を決めやすい
大きさの変更	ページを足して対応	セグメントの長さを実行中に変えやすい

R6秋 問18：ページングの長所は「**外部断片化が発生しない**」。論理単位での保護・共用・大きさの動的変更は、どれも**セグメンテーション**の長所。

R4秋 問17：ページサイズを半分にすると

大半のプログラムがページの半分以下



1ページの中の使わない部分（内部断片化）が減る

→ **主記憶にたくさん載り、主記憶の不足が和らぐ（正解ウ）**

逆に、ページを小さくするとページテーブルは大きくなる。「ページフォールトが増えて遅くなる」「余裕がなくなる」は、この問題の前提では誤り。

フォールト=ページイン ≧ ページアウト。平均時間は割合で重み付け

ページフォールト
ページが主記憶にない
▶
ページアウト
空きがなければ追い出す
▶
ページイン
補助記憶から読む

R3春 問19：フォールトが起きたら必ずページインするので**フォールト=ページイン**。ページアウトは空きがない時だけ（追い出すページが更新されていない時は書き戻しも不要）なので**それ以下**。答えは「PF=PI ≧ PO」。

ページアウトの対象をどれにするかが、次のスライドのページ置換えアルゴリズム。

R5秋 問16：平均処理時間30ミリ秒。ページアウトなしの割合は？

$$20 \times p + 60 \times (1 - p) = 30$$
$$60 - 40p = 30 \quad \rightarrow \quad p = 0.75$$

ページアウトなし20ミリ秒、あり60ミリ秒

R3秋 問17：許されるページフォールトの発生率

$$\text{1命令の遅れ} = 2\text{回} \times \text{率} \times 40\text{ミリ秒} \leqq 0.4\text{マイクロ秒}$$
$$\text{率} \leqq 0.4\times10^{-6} \div (80\times10^{-3}) = 5\times10^{-6}$$

ミリ（10^{-3}）とマイクロ（10^{-6}）の桁をそろえてから割る。「1命令で2回参照」の2を落とすと1×10^{-5}（イ）になる。

FIFOは「一番先に読み込んだページ」を出す。途中で使ったかは関係ない

R5春 問17：3つの枠、参照順 1, 4, 2, 4, 1, 3（枠の位置は動かない）

参照	枠1	枠2	枠3	結果
1	1			読込み
4	1	4		読込み
2	1	4	2	読込み
4	1	4	2	あり
1	1	4	2	あり
3	3	4	2	置換え（1を出す）

3を読む時、読み込んだ時刻が一番古いのは**ページ1**（直前に使ったばかりでも出す）。枠1が3になり、最後は **3, 4, 2**（ウ）。

方式	追い出すページ	見る情報
FIFO	主記憶に**最も早く読み込んだ**ページ	読み込んだ時刻
LRU	**最後に使ってから最も時間がたった**ページ	最後に使った時刻
LFU	**使った回数**が最も少ないページ	使った回数
LIFO	最も新しく読み込んだページ	読み込んだ時刻
OPT（最適）	この先**最も長く使わない**ページ	未来の参照（実際には作れない比較の基準）

FIFOでは、枠の数を増やしたのにフォールトが増えることがある（**ビレディの逆説**）。

⏳ LRUは「最後に使った時刻」が一番古いページを出す。使うたびに時刻を更新

R7春 問15：4つの枠（4000〜7000番地）

参照順 1, 2, 3, 4, 2, 5, 3, 1, 6, 5, **4**。最後の4はどこに入る？

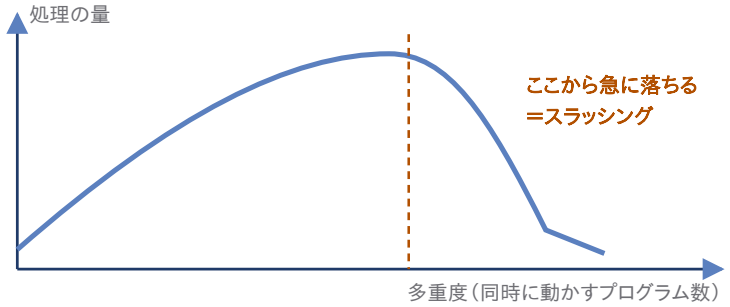
参照	4000	5000	6000	7000	結果
1〜4	1	2	3	4	読込み
2	1	**2**	3	4	あり
5	**5**	2	3	4	**置換え（1）**
3	5	2	**3**	4	あり
1	5	2	3	**1**	**置換え（4）**
6	5	**6**	3	1	**置換え（2）**
5	**5**	6	3	1	あり
4	5	6	**4**	1	**置換え（3）**

参照	その時の「最後に使った順」（古い→新しい）	出す
5	1, 3, 4, 2	1
1	4, 2, 5, 3	4
6	2, 5, 3, 1	2
4	3, 1, 6, 5	**3**

最後の4はページ3がいた**6000番地**（ウ）。「2」「3」「5」を使い直すたびに、その順番が一番新しい側へ動くのがFIFOとの違い。

R5春 問18：LRUは「**使用されてからの経過時間が最も長い**ページを置き換える」。使用回数で選ぶのはLFU。

♻ ページの出し入ればかりで急に遅くなるのがスラッシング。多重度を下げて防ぐ



R3秋 問16・R6秋 問17（同じ問題）：「ページの置換えが頻繁に起きて、処理能力が急に落ちる」のは**スラッシング**。スワップアウト・フラグメンテーション・ページフォールトは、それぞれ1つの出来事や状態の名前。

R7秋 問19：キャッシュの表（読み込んだ時刻・最後に参照した時刻・参照回数）

選択肢	表のどの列で選ぶか
FIFO	読み込んだ時刻が**最も古い**もの
LFU	参照回数が**最も少ない**もの
LIFO	読み込んだ時刻が**最も新しい**もの
LRU	最後に参照した時刻が**最も古い**もの（この問題の答え）

キャッシュメモリの置換え（#3の範囲）も、ページ置換えと同じ名前と考え方。**列を1つずつ当てはめて**、C2が選ばれる方式を探す。

第4章

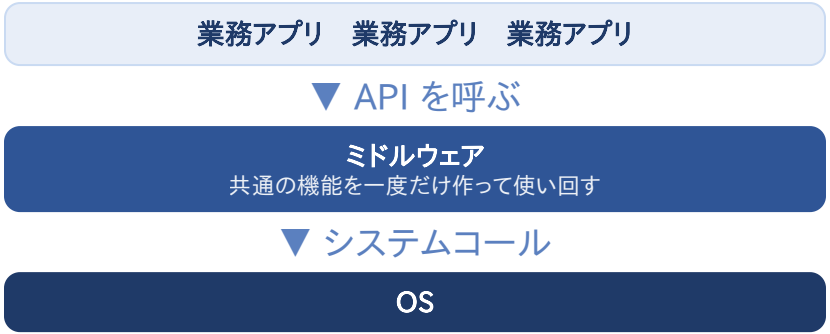
ミドルウェアとファイルシステム

ミドルウェア ／ API と ライブラリ ／ シェル ／ ディレクトリとパス ／ リンク ／ バックアップ ／ ファイル編成



🤝 ミドルウェアは多くのアプリに共通の機能。OSとアプリの間に入る

ミドルウェア	受け持つこと
DBMS	データベースの定義・操作・同時実行の制御（#9）
TPモニター	大量のトランザクションを受け付け、処理を振り分けて完了まで管理する
アプリケーションサーバ	Webアプリの業務処理を動かす土台
メッセージキュー（MOM）	アプリ間でメッセージを**ためて非同期に**受け渡す
運用管理ツール	ジョブの自動実行・稼働の監視・ログの収集
ORB（CORBA）	ネットワーク上の別のオブジェクトを呼び出す



ミドルウェアを使うと、アプリは**OSの違いを気にせず**作れて、開発の手間も減る。Webサーバ・DBMS・アプリケーションサーバの3層は、#4の3層クライアントサーバと対応する。

🔗 APIは呼び出し方の約束。ライブラリは「いつつなぐか」で静的と動的

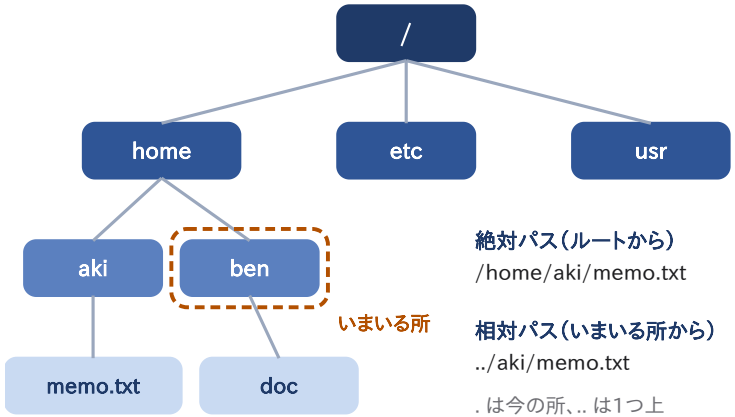
言葉	意味
API	ソフトウェアの機能を呼び出すための**決まりごと**（関数名・引数・戻り値）。ソースコードの段階の約束
ABI	機械語の段階の約束（呼出し規約・データの並べ方）。同じなら作り直さずに動く
Web API	HTTPで呼び出すAPI。REST方式が多い
OpenAPI	Web APIの仕様を書く形式。**R7春 問17**：それに沿って定義・開発を支えるOSSは**Swagger**

シェル（bashなど）は、利用者が打ったコマンドを解釈してOSに伝える**コマンドインタプリタ**。複数のコマンドを並べたファイルが**シェルスクリプト**。

	静的リンク	動的リンク
つなぐ時	実行ファイルを作る時（リンカ）	**実行する時**（DLL・共有ライブラリ）
実行ファイル	ライブラリ入りで大きい	小さい。複数のプログラムで共用
ライブラリの更新	作り直しが必要	差し替えるだけで反映

ライブラリは**自分のコードから呼ぶ**部品。フレームワークは土台の側が**自分のコードを呼ぶ**（制御の反転）。

ファイルは木の形で管理する。パスはルートからか、今いる所から書く



絶対パス（ルートから）
/home/aki/memo.txt

相対パス（いまいる所から）
../aki/memo.txt

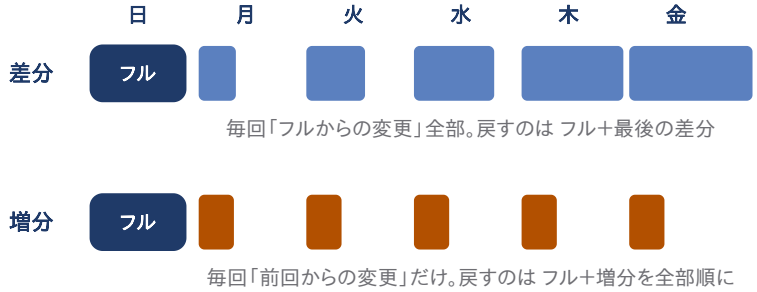
. は今の所、.. は1つ上

リンク	指しているもの	元を消すと
ハードリンク	同じ**ファイルの実体**（iノード）にもう1つ名前を付ける	残りの名前から**まだ読める**
シンボリックリンク	元のファイルの**パス名**を中身に持つ別ファイル（ショートカット）	指す先がなくなり**読めない**

> シンボリックリンクはディレクトリや別のファイルシステムも指せる。ハードリンクは同じファイルシステムの中だけ。

> アクセス権（UNIX）：所有者・グループ・その他 ごとに**読み r・書き w・実行 x**。`rwxr-x---` は所有者が全部、グループが読みと実行、その他は何もできない。

増分は取るのが速く戻すのが遅い。差分はその中間



方式	取る時間・容量	戻す手間
フル	最大	1つ戻すだけ
差分	日がたつほど増える	フル＋最新の差分の**2つ**
増分	最小	フル＋増分**全部**（1つ欠けると戻せない）

ファイルの編成	しくみ
順編成	先頭から順に並べる。順に読むのは速い
直接編成	キーから**計算で**位置を決める（ハッシュ）。1件を探すのが速い
索引順編成	順に並べた本体＋キーの**索引**。順にも1件ずつにも読める
区分編成	複数の順編成のメンバと、その場所を示す登録簿。プログラムの保管

ファイルシステムの種類：WindowsのNTFS、Linuxのext4、USBメモリ向けのFAT32・exFAT。書込みの記録（ジャーナル）を先に残すジャーナリングで、停電後も壊れにくくする。



第5章

開発ツールとOSS

コンパイラの処理 ／ リンカとローダ ／ 言語処理の種類 ／ デバッグとテストの道具 ／ バージョン管理 ／ OSSのライセンス ／ 用途別のOSS



コンパイラは字句→構文→意味→最適化→コード生成。生成の直前が最適化

字句解析
文字の列を単語（トークン）に

▶

構文解析
文法どおりか・構文木

▶

意味解析
型や宣言の矛盾

▶

最適化
速く・小さく

▶

コード生成
目的プログラム

コンパイラ
原始→目的プログラム

▶

リンカ
→実行可能プログラム

▶

ローダ
主記憶に載せる

リンカ（連係編集）は目的プログラムとライブラリを**つなぐ**。ローダは実行可能プログラムを主記憶に**載せる**。載せる場所に合わせて番地を直すのが**再配置**。

R6秋 問20：コード生成の**直前**に行うのは**最適化**。字句解析は最初、構文解析と意味解析は途中。

最適化の例	中身
定数の畳込み	2*3 を先に 6 と計算しておく
ループ不変式の移動	ループの中で変わらない計算を外に出す
インライン展開	小さな関数の呼出しを中身で置き換える

別のCPU向けの機械語を作るのがクロスコンパイラ。組込み開発の基本

言語プロセッサ	何をするか
コンパイラ	原始プログラムを**まとめて**目的プログラムに翻訳
インタプリタ	1文ずつ解釈しながら**すぐ実行**。目的プログラムを作らない
アセンブラ	アセンブリ言語（機械語と1対1）を機械語に
クロスコンパイラ	動かすのと**異なる命令形式**のコンピュータ用の目的プログラムを作る
JITコンパイラ	実行中に、よく使う中間コードを機械語へ直す（Java・JavaScript）
プリプロセッサ	コンパイルの前に、マクロの展開やファイルの取込みを行う



R5秋 問18:「異なる命令形式のコンピュータで実行できる目的プログラムを生成する」のは**クロスコンパイラ**。エミュレーター（他の機械の**まね**をして動かす）、最適化コンパイラ、プログラムジェネレーター（仕様から**プログラムを自動生成**）と入れ替えた選択肢に注意。

開発するコンピュータを**ホスト**、動かす機器を**ターゲット**と呼ぶ。

回数と時間を測るのがプロファイラ。全履歴を手元に持つのがGit

道具	何をするか
プロファイラ	関数などの**実行回数・実行時間**を集め、遅い所を見つける
デバッガ	止めながら実行し、変数の中身を見て誤りを探す（ブレークポイント・トレース）
エミュレーター	別のハードウェアの動きを**まねて**、そのプログラムを動かす
シミュレーター	実物の代わりに、ソフトウェアで**動きを模擬**して確かめる
テストドライバ・スタブ	下位を試す時の**呼ぶ側の代わり**・上位を試す時の**呼ばれる側の代わり**（#12）
静的解析ツール	動かさずにソースコードを調べて誤りや規約違反を探す

R6春 問19：実行回数や実行時間を集めるのは**プロファイラ**。

	集中型	分散型
例	Subversion・CVS・RCS	**Git**・Mercurial
履歴の置き場所	サーバに1つ	各自の手元に**全履歴の完全な複製**
つながらない時	記録（コミット）できない	手元でコミットでき、あとで送る（push）

R3秋 問18・R6秋 問19（同じ問題）：「ローカルに全履歴の完全な複製を持つ」のは**Git**。ほか3つは集中型（RCSは1ファイル単位の古い道具）。

📜 ソース公開の義務は GPL→LGPL→MPL→BSD の順に軽くなる

R7春 問16：改変して配るとき、ソースの公開が必要か			
ライセンス	改変した部分	静的リンクした自分のコード	動的リンクした自分のコード
GPL	必要	必要	必要
LGPL	必要	必要（結合部分の再リンクを許す）	不要
MPL	必要（ファイル単位）	不要	不要
BSD・MIT・Apache	不要	不要	不要

表の○の数が多い順に **GPL, LGPL, MPL, BSD**（ア）。GPLのように「派生物も同じ条件で配る」考え方を**コピーレフト**と呼ぶ。

OSI の定義	中身
自由な再配布	売ってもよい。ライセンス料を取らない
ソースコード	入手できるようにする
差別の禁止	人・団体・**利用分野**で制限しない（「商用は禁止」は不可）

OSSでも**著作権は放棄されていない**。無保証が原則で、ライセンスの条件を守る義務がある。

OSSは用途とセットで覚える。似た分野の名前と入れ替えて出る

用途	代表的なOSS	過去問で問われ方
OS	Linux・FreeBSD・Android	R5秋19：Linuxカーネルの役目
仮想化・コンテナ	**Docker**（コンテナ）・Kubernetes（コンテナの管理）・KVM・Xen・QEMU（仮想マシン）	R5春20：コンテナ型仮想化は**Docker**
大規模データ	**Hadoop**（分散処理）・Spark	R3春20：大量データを分散処理するライブラリ
Web・API	Apache HTTP Server・nginx・Tomcat（アプリサーバ）・**Swagger**（API定義）・curl	R7春17：OpenAPIに従った定義はSwagger
DBMS	MySQL・PostgreSQL・SQLite・Redis・MongoDB	—
運用・構成管理	Ansible・Chef（構成の自動化）・Serverspec（構成のテスト）・Zabbix（監視）・Fluentd（ログ収集）	Serverspecは選択肢に登場
認証	OpenAM・Keycloak（シングルサインオン）	OpenAMは選択肢に登場
開発	Git・Jenkins（CI）・Eclipse・GCC（コンパイラ）	R3秋18：Git



📋 まとめ：表と線を自分で書けるようにすれば、42問の論点はほぼ取れる

論点	覚えること	過去問
状態遷移	横取り→実行可能。入出力の開始→待ち。待ち→実行は直接移らない	R3春17・R7春14・R6春16
割込みとRTOS	割込み処理は最高優先度のタスクより先。周期タスクは高い方の使用時間を引く	R4秋18・R5秋17
時間の計算	線を引く。ターンアラウンド＝終了－到着。多重度は到着のずれに注意	R3春16・R7秋15・R4春19・R6春18
スケジューリング	SJFはスタベーション、エージングで防ぐ	R6秋16
排他制御	共有資源はセマフォ。デッドロックは取る順番をそろえる。スレッドセーフ	R4春17・R7秋16・R4秋16・R6春17・R7秋17
記憶管理	断片化＝連続した空きがない。ページングは外部断片化なし。GC	R3春18・R4春18・R6秋18・R4秋17
仮想記憶	PF＝PI≧PO。FIFO＝読込み時刻、LRU＝最終使用時刻。スラッシング	R3春19・R5春17・R7春15・R3秋16 ほか
開発ツール・OSS	最適化はコード生成の直前。クロスコンパイラ・プロファイラ・Git。GPL→BSD	R6秋20・R5秋18・R6春19・R7春16 ほか